

מודול 3 — יסודות רשתות

חומר לימוד

1 יחידות

תוכן העניינים

1. מודול 3 — יסודות רשתות (Networking Fundamentals)

■ הכול בגלילה אחת: כל החומר של המודול בקובץ אחד — חומר לימוד, תרגילים, תרגול ופתרונות, ברצף.

מודול 3 — יסודות רשתות (Networking Fundamentals)

מטרות המודול: להבין לעומק כיצד מחשבים מתקשרים ברשת — מודל OSI ו-TCP/IP, מול UDP ולחיצת היד, כתובות MAC, IP (IPv4/IPv6) ו-ARP, פורטים ופרוטוקולים, Subnetting, ופרוטוקולי התשתית (DNS, DHCP, NAT, ICMP). כל זה עם דוגמאות מעשיות ופקודות.

קבצים: `slides.md` · `practice.md` · `solutions.md` · `missions.md` · `README.md`

🎬 **פרק זה מלא בהדגמות חיות!** אחרי כל נושא יש בלוק “**הדגמה** — **נסה בעצמך**” עם הוראות **צעד-אחר-צעד** להרצה על Kali. בין השאר: שרת Flask שאתם פותחים וסורקים, לחיצת יד ב-tcpdump, ARP חי, ו-nmap על הרשת. **לומדים בעשייה — לא רק בתיאוריה.**

מבוסס על שיעור המקור #07 (“Networking — 3 hours”).

למה זה חשוב להאקר? כל תקיפה עוברת דרך הרשת. בלי להבין פורטים, פרוטוקולים, כתובות ולחיצת יד — אי אפשר להבין מה סורקים, מה מוצאים, וכיצד מנצלים. זה הבסיס למודולים 5–6 (סריקה וניצול).

תוכן העניינים

1. מהי רשת ומסע החבילה
2. מודל OSI
3. מודל TCP/IP
4. TCP מול UDP ולחיצת היד
5. כתובות IP
6. MAC ו-ARP
7. פורטים ופרוטוקולים
8. Subnetting
9. פרוטוקולי תשתית: DNS, DHCP, NAT
10. כלים מעשיים

3.1 מהי רשת ומסע החבילה

רשת היא אוסף מכשירים המתקשרים ביניהם. המידע נשלח **בחבילות (Packets)** — יחידות קטנות של נתונים העוברות מהמקור ליעד, לעיתים דרך צמתים רבים (Routers).

מסע חבילה בקצרה — מה קורה כשאתה גולש ל- `example.com`

1. **DNS**: המחשב שואל את שרת ה-DNS "מה הכתובת של `example.com`?", ומקבל בחזרה כתובת IP — למשל `93.184.216.34`.
2. **ARP**: כדי לשלוח ברשת המקומית, המחשב מוצא את כתובת ה-MAC של ה-Gateway.
3. **TCP Handshake**: נפתח חיבור לפורט `80` או `443` (הרצף `SYN → SYN-ACK → ACK`).
4. **HTTP**: נשלחת בקשה `GET /`, והשרת מחזיר את הדף.

💡 כל שלב כאן שייך לשכבה אחרת במודל OSI — וכל שלב הוא גם **נקודת תקיפה אפשרית**: זיוף DNS, הרעלת ARP, סריקת פורטים, וחולשות Web.

הדגמה — נסה בעצמך: מסע החבילה

עקוב **צעד-אחר-צעד**. אל תדלג — כל צעד בונה על הקודם.

צעד 1 — עקוב אחרי חבילה עד יעד באינטרנט (דורש מצב רשת NAT):

```
tracert -n 8.8.8.8
```

👁️ **מה תראה**: שורות ממוספרות `1, 2, 3...` — כל שורה היא **ראוטר** (תחנה) שהחבילה עברה דרכו, עם כתובת ה-IP שלו וזמן התגובה.

צעד 2 — אין אינטרנט? עקוב עד ה-Gateway המקומי:

```
ip route | grep default      # Gateway-מצא את כתובת ה-
tracert -n <Gateway-כתובת> # הדבק אותה כאן
```

👁️ **מה תראה**: תחנה אחת — ה-Gateway של הרשת שלך.

🎯 **מה קרה כאן**: החבילה לא "קפצה" ליעד — היא עברה **מראוטר לראוטר**. הרגע ראית את המסע שלה בשידור חי.

🧠 **בדוק את עצמך:** מה מייצגת כל שורה בפלט של `tracert` (א - פורט פתוח על היעד - ב) ראוטר (תחנה) בדרך ליעד - ג) חבילה שאבדה

✅ **תשובה: ב** — כל שורה היא צומת/ראוטר שהחבילה עברה דרכו במסע ליעד.

3.2 מודל OSI

מודל **OSI** מתאר תקשורת בשבע שכבות. כל שכבה מדברת עם המקבילה בצד השני, ומוסיפה/מסירה “עטיפה” משלה.

#	שכבה	תפקיד	דוגמאות	תקיפה אופיינית
7	Application	ממשק לאפליקציה	HTTP, DNS, FTP, SSH	חולשות Web, פישונג
6	Presentation	הצפנה/קידוד	TLS/SSL	פענוח תעבורה
5	Session	ניהול חיבורים	RPC, NetBIOS	חטיפת Session
4	Transport	תעבורה מקצה-לקצה	TCP, UDP	סריקת פורטים
3	Network	ניתוב וכתובות לוגיות	IP, ICMP	Spoofing, ניתוב
2	Data Link	כתובות פיזיות	Ethernet, ARP, MAC	ARP Poisoning
1	Physical	אותות וכבלים	Wi-Fi, כבל	האזנה פיזית

💡 **טריק לזכירה (מלמעלה למטה):** *All People Seem To Need Data Processing*

דוגמה להמחשה: כשאתה שולח מייל — שכבה 7 יוצרת את ההודעה, שכבה 4 מחלקת אותה לחבילות TCP, שכבה 3 מוסיפה כתובות IP, שכבה 2 מוסיפה כתובות MAC, ושכבה 1 שולחת את הביטים. בצד השני התהליך הפוך.

🎬 הדגמה — נסה בעצמך: לראות את השכבות בבקשה אחת

נשלח בקשת HTTP אחת ונזהה בתוכה את השכבות.

צעד 1 — הפעל שרת מקומי (טרמינל 1) כדי לעבוד בלי אינטרנט:

```
pip install flask          # פעם אחת בלבד
python3 demo_server.py     # משאיר רץ - פותח פורט 5000
```

צעד 2 — שלח בקשה “מדברת” (טרמינל 2):

```
curl -v http://127.0.0.1:5000
```

👁️ מה תראה (השורות החשובות):

```
* Connected to 127.0.0.1 port 5000  ← פורט + IP שכבות 3+4: כתובת
> GET / HTTP/1.1                  ← שכבה 7: בקשת האפליקציה
< HTTP/1.1 200 OK                 ← שכבה 7: התשובה מהשרת
```

🎯 מה קרה כאן: בקשה אחת “ירדה” דרך השכבות — מצאה כתובת (שכבה 3), פתחה פורט (שכבה 4), ושלחה HTTP (שכבה 7). ראית את זה בפלט.

🧠 בדוק את עצמך: השורה `> GET / HTTP/1.1` שראית ב-`curl -v` שייכת לאיזו שכבה? (א - שכבה 3 (Network) - ב) שכבה 4 (Transport) - ג) שכבה 7 (Application)

✅ תשובה: ג — HTTP הוא פרוטוקול אפליקציה, שכבה 7.

3.3 מודל TCP/IP

TCP/IP הוא הגרסה המעשית (4 שכבות) שרשת האינטרנט משתמשת בה בפועל. הוא “מקפל” את שכבות OSI:

TCP/IP	מקביל ל-OSI	דוגמאות
Application	5–7	HTTP, DNS, SSH
Transport	4	TCP, UDP
Internet	3	IP, ICMP
Network Access	1–2	Ethernet, ARP

🎬 הדגמה — נסה בעצמך: לכל שכבה יש פקודה

(השרת מ-3.2§ עדיין רץ? מצוין.)

צעד 1 — שכבות 2-3 (כתובות):

```
ip a
```

👁👁 link/ether = כתובת **MAC** (שכבה 2); inet = כתובת **IP** (שכבה 3).

צעד 2 — שכבה 4 (פורטים וחיבורים):

```
ss -tunp
```

👁👁 חיבורים פעילים עם מספרי פורט — שכבת התעבורה.

צעד 3 — שכבה 7 (אפליקציה):

```
curl 127.0.0.1:5000
```

👁👁 תוכן טקסטואלי — שכבת האפליקציה.

🎯 **מה קרה כאן:** ארבע השכבות אינן מופשטות — **לכל אחת כלי משלה**, והרגע נגעת בכולן.

🧠 **בדוק את עצמך:** איזו פקודה מציגה את **שכבה 4** (פורטים וחיבורים)? (א - ip a (ב - ss -tunp (ג -

```
curl
```

✅ **תשובה: ב** — ss מציג חיבורים ופורטים (שכבת התעבורה). ip a = שכבה 2-3, curl = שכבה 7.

7.

3.4 TCP מול UDP ולחיצת היד

בשכבת התעבורה שני פרוטוקולים עיקריים:

	TCP	UDP
אמינות	אמין — מוודא הגעה	לא אמין — “יורה ושוכח”
חיבור	Connection-oriented	Connectionless
מנגנון	לחיצת יד תלת-שלבית	ללא
מהירות	איטי יותר	מהיר יותר
שימוש	web, מייל, קבצים	DNS, סטרימינג, VoIP

לחיצת היד התלת-שלבית (3-Way Handshake)

כך נפתח כל חיבור TCP — וזהו הבסיס לסריקת פורטים:

לקוח	---	SYN	---	>	(בקשת חיבור)	שרת
לקוח	<---	SYN-ACK	--		(אישור + בקשה חוזרת)	שרת
לקוח	---	ACK	---	>	(החיבור פתוח)	שרת

קשר ישיר לתקיפה — כך nmap קובע אם פורט פתוח: | תשובת השרת ל-SYN | מסקנה | |

SYN-ACK | |-----|-----| | הפורט פתוח | | (Reset) RST | הפורט סגור | | אין תשובה | מסונן

(Filtered — חומת אש) |

סריקת “SYN Scan” (nmap -sS) שולחת SYN ולא משלימה את הלחיצה — מהירה ו”שקטה” יותר.

הדגמה — נסה בעצמך: לתפוס את לחיצת היד בשידור חי ⭐

ההדגמה הכי מרשימה בפרק. נראה את SYN → SYN-ACK → ACK קורים באמת. ודא ששרת ה-Flask רץ (מ-3.28).

צעד 1 — טרמינל 1: התחל להאזין לתעבורה על פורט 5000:

```
sudo tcpdump -i lo -n 'tcp port 5000'
```

👁 מופיע listening on lo... — הוא ממתין.

צעד 2 — טרמינל 2: צור חיבור:

```
curl http://127.0.0.1:5000
```

צעד 3 — חזור לטרמינל 1 וקרא את הפלט:

```
Flags [S]    ← SYN      ("הלקוח: "אפשר להתחבר")
Flags [S.]   ← SYN-ACK   ("השרת: "כן, וגם אני מבקש")
Flags [.]    ← ACK       ("הלקוח: "מאושר!" – החיבור פתוח)
Flags [P.]   ← הנתונים  (HTTP-בקשת ה)
Flags [F.]   ← FIN       (סגירה מסודרת)
```

🎯 מה קרה כאן: ה-Way Handshake-3 שראית בדיאגרמה — קרה מול העיניים שלך. עצור עם Ctrl+C.

בונוס — UDP בלי לחיצת יד:

```
sudo tcpdump -i any -n udp port 53 # 1 טרמינל
dig google.com                     # 2 (NAT דורש) טרמינל
```

👁️ DNS רץ על UDP — שאלה, תשובה, בלי SYN/ACK. השוואה חיה ל-TCP.

🧠 בדוק את עצמך: מהו הדגל הראשון (Flags [S]) שראית ב-tcpdump? (א - ACK - ב) SYN - ג) FIN

✅ תשובה: ב — [S] = SYN, הצעד הראשון בלחיצת היד (הלקוח מבקש להתחבר).

3.5 כתובות IP

כתובת IP מזהה מכשיר ברשת. שתי גרסאות: **IPv4**: ארבעה מספרים (0–255), למשל 192.168.1.10. **IPv6**: פורמט עשירי (Decimal). 32 סיביות, בפורמט עשירי (Decimal). 128 סיביות, למשל 2001:0db8:85a3::8a2e:0370:7334. בפורמט הקסדצימלי — נועד לפתור את מחסור הכתובות.

פרטי מול ציבורי

- **פרטי (Private)** — לשימוש פנימי, אינו מנותב באינטרנט:
 - 10.0.0.0 – 10.255.255.255 (/8)
 - 172.16.0.0 – 172.31.255.255 (/12)
 - 192.168.0.0 – 192.168.255.255 (/16)
- **ציבורי (Public)** — ייחודי וגלוי באינטרנט.
- **Loopback** — 127.0.0.1 = המכונה עצמה (localhost).

💡 כשתראה 192.168.x.x או 10.x.x.x בבדיקה — אתה ברשת פנימית. זה מכתیب את הצעד הבא (Pivoting — מודול 12).

🎬 הדגמה — נסה בעצמך: הכתובת שלך מול איך שהעולם רואה אותך

צעד 1 — הכתובת הפרטית שלך:

```
ip -c a
```

👁️ שורת inet תחת eth0 — למשל 192.168.56.10 . זו כתובת פרטית.

צעד 2 — הכתובת הציבורית (דורש NAT):

```
curl -s ifconfig.me; echo
```

👁️ כתובת שונה לגמרי — זו מה שכל האינטרנט רואה.

צעד 3 — loopback (המכונה מדברת עם עצמה):

```
ping -c2 127.0.0.1
```

🎯 מה קרה כאן: יש לך כתובת פרטית ברשת המקומית, אבל כלפי האינטרנט מופיעה כתובת ציבורית אחת — זהו ה-NAT (נראה אותו ב-3.9).

🧠 בדוק את עצמך: הכתובת 192.168.56.10 היא — (א - ציבורית - ב) פרטית - ג) Loopback

✅ תשובה: ב — 192.168.x.x הוא טווח פרטי. (Loopback = 127.0.0.1)

ARP-1 MAC 3.6

MAC Address

כתובת פיזית קבועה של כרטיס הרשת, בשכבה 2.

- פורמט: 00:1A:2B:3C:4D:5E
- “צרוכה” בחומרה — אך ניתנת לזיוף (טכניקת MAC Spoofing).

ARP — Address Resolution Protocol

פרוטוקול שמתרגם כתובת IP לכתובת MAC ברשת המקומית.

איך זה עובד: כשמכונה רוצה לשלוח מידע לכתובת מקומית, היא "צועקת" ברשת — "מי מחזיק את הכתובת הזו?" — והבעלים עונה בחזרה עם כתובת ה-MAC שלו.

ARP Poisoning

מתקפת "איש בתווך" (Man-in-the-Middle) בשכבה 2.

התוקף עונה לבקשות ARP בשקר, וגורם לכל התעבורה לעבור דרכו — וכך הוא מאזין לתקשורת. זו מתקפה קלאסית ברשת מקומית.

פקודות לצפייה בטבלת ה-ARP:

```
ip neigh      # ידוע MAC <-> IP מיפוי
arp -a        # תחביר ישן
```

הדגמה — נסה בעצמך: ARP בשידור חי

צעד 1 — נקה את טבלת ה-ARP:

```
sudo ip neigh flush all
ip neigh                # הטבלה ריקה עכשיו
```

צעד 2 — "אלץ" את המכונה לשאול מי ה-Gateway:

```
ping -c1 $(ip route | grep default | awk '{print $3}')
ip neigh                # הופיע Gateway-של ה-MAC עכשיו
```

👁️ שורה חדשה: `<gateway-ip> ... lladdr <MAC> REACHABLE`.

צעד 3 — לראות את השיחה עצמה (אם יש רשת מקומית, שני טרמינלים):

```
sudo tcpdump -i eth0 arp
# למכונה אחרת ברשת — תראה ping ואז:
# ARP, Request who-has X tell Y    ///    ARP, Reply X is-at <MAC>
```

🎯 **מה קרה כאן:** ARP הוא "ספר הטלפונים" של הרשת המקומית — ממפה IP → MAC. ראית אותו מתמלא בזמן אמת.

🧠 **בדוק את עצמך:** מה **ARP** ממפה? (א - שם דומיין → IP - ב) IP → MAC - ג) פורט → שירות

✅ **תשובה: ב** — ARP מתרגם כתובת IP לכתובת MAC ברשת המקומית. (שם → IP זה DNS).

3.7 פורטים ופרוטוקולים נפוצים

פורט (Port) הוא "דלת" ממוספרת (0–65535) שדרכה שירות מקבל תקשורת. זיהוי הפורטים הפתוחים = זיהוי השירותים = **משטח התקיפה**.

פורט	פרוטוקול	שירות	הערה לתקיפה
21	TCP	FTP	העברת קבצים, לרוב לא מוצפן
22	TCP	SSH	גישה מרוחקת מאובטחת; יעד Brute Force
23	TCP	Telnet	גישה לא מוצפנת — מסוכן
25	TCP	SMTP	דואר יוצא
53	TCP/UDP	DNS	תרגום שמות
80	TCP	HTTP	web (לא מוצפן)
110	TCP	POP3	קבלת דואר
139/445	TCP	SMB	שיתוף קבצים Windows — יעד מרכזי
143	TCP	IMAP	קבלת דואר
443	TCP	HTTPS	web מוצפן (TLS)
3306	TCP	MySQL	בסיס נתונים
3389	TCP	RDP	שולחן עבודה מרוחק Windows

טווחי פורטים: **Dynamic** , (1024–49151) **Registered** , (0–1023) **Well-Known** (49152–65535) .

💡 שגן לפחות: 21, 22, 80, 443, 445, 3389 — הם חוזרים כמעט בכל בדיקה.

🎬 **הדגמה מרכזית — נסה בעצמך: פתח פורט וסרוק אותו** 🚀

זו הליבה של הפרק. נפעיל שרת, "נראה" את הפורט נפתח, נסרוק אותו, ונסגור אותו.

צעד 1 — התקן Flask (פעם אחת):

```
pip install flask
```

צעד 2 — טרמינל 1: הפעל את השרת (הקובץ `demo_server.py` בתיקיית המודול):

```
python3 demo_server.py
```

👁️ השרת מדפיס שהוא מאזין על פורט 5000. **השאר אותו רץ.**

צעד 3 — טרמינל 2: "ראה" את הפורט הפתוח:

```
ss -tulpn | grep 5000
```

👁️ שורה עם LISTEN ו-0.0.0.0:5000 — הפורט פתוח!

צעד 4 — סרוק אותו כמו האקר:

```
nmap 127.0.0.1 -p 5000
```

👁️ 5000/tcp open .

צעד 5 — התחבר לשירות:

```
curl http://127.0.0.1:5000/whoami
```

👁️ השרת עונה עם ה-IP שלך. (נסה גם בדפדפן `http://127.0.0.1:5000` — יש דגל נסתר ב-`/secret` !)

צעד 6 — סגור את הדלת: חזור לטרמינל 1, לחץ `Ctrl+C`, ואז:

```
ss -tulpn | grep 5000      # רייק - הפורט נסגר
```

🎯 **מה קרה כאן:** פורט הוא פשוט **דלת ששירות פותח**. שירות רץ = דלת פתוחה, ולכן `nmap` מוצא אותה. עצרת = הדלת נסגרה. זה כל הרעיון של "סריקת פורטים".

🏆 **פעילות כיתתית — "סרוק את השכן":** 1. הרץ את השרת (הוא מאזין לכל הרשת — 0.0.0.0). 2. מצא את ה-IP שלך: `ip a`. 3. תן אותו לשכן — והוא ירוץ: `nmap -p 5000 <שלך-IP>`. 🙌 סרקתם אחד את השני — בדיוק כמו בבדיקת חדירה אמיתית!

בונוס — מטרה עם 5 פורטים (בלי התקנות; הקובץ `multiport_server.py`):

```
python3 multiport_server.py          # טרמינל 1
nmap 127.0.0.1 -p 2121,2323,8080,9000,13337 # טרמינל 2 - 5 פורטים פתוחים
nc 127.0.0.1 8080                     # של השירות המדומה "Banner" תופס"
```

👁️ סריקה שנראית כמו מטרה אמיתית עם כמה שירותים.

🧠 **בדוק את עצמך:** אחרי שעצרת את השרת (Ctrl+C), למה `ss` כבר לא מצא את פורט 5000? (א) הפורט תמיד סגור - (ב) השירות שפתח אותו הפסיק לרוץ - (ג) `nmap` מחק אותו

✅ **תשובה: ב** — פורט פתוח רק כל עוד השירות שמאזין עליו רץ. עצרת את השירות → הדלת נסגרה.

3.8 Subnetting (חלוקת רשת)

Subnetting הוא חלוקת רשת לתת-רשתות. עבור בודק חדירות, המטרה המעשית: **להבין את גבולות הרשת** — אילו כתובות שייכות לאותה תת-רשת (ולכן ייסרקו יחד).

מסכת רשת ו-CIDR

- **מסכת רשת (Subnet Mask)** מפרידה בין חלק הרשת לחלק המארח (Host).
- **CIDR** מקצר זאת: המספר אחרי `/` = כמה סיביות שייכות לרשת.

CIDR	מסכה	מס' כתובות	מארחים שמישים	דוגמה
<code>/24</code>	255.255.255.0	256	254	<code>192.168.1.0/24</code>
<code>/16</code>	255.255.0.0	65,536	65,534	<code>172.16.0.0/16</code>
<code>/8</code>	255.0.0.0	16.7M~	16.7M~	<code>10.0.0.0/8</code>

🔍 **דוגמה מעשית:** `nmap 192.168.1.0/24` = "סרוק את כל 256 הכתובות בתת-הרשת". `ipcalc`
`192.168.1.0/24` יראה לך את הכתובת הראשונה, האחרונה וה-Broadcast.

🎬 **הדגמה — נסה בעצמך: ראה כל מכשיר ברשת**

צעד 1 — התקן ipcalc:

```
sudo apt install ipcalc -y
```

צעד 2 — חשב את גבולות תת-הרשת:

```
ipcalc 192.168.1.0/24
```

החלף בטווח שלך

👁️ רואים: Netmask, כתובת Broadcast, ו-254 Hosts/Net.

צעד 3 — גלה מי חי ברשת (Host Discovery):

```
nmap -sn 192.168.1.0/24
```

(ip a: בדוק עם) החלף בטווח שלך

👁️ רשימת כל המכשירים החיים ברשת — מחשבים, טלפונים, הראוטר.

🎯 **מה קרה כאן:** /24 אינו תיאוריה — אלה 254 הכתובות שסורקים יחד, ו-nmap -sn הראה אילו מהן קיימות באמת ברגע זה.

🧠 **בדוק את עצמך:** כמה מארחים שמישים יש בתת-רשת /24? (א - 256 - ב) 254 - ג) 512

✅ **תשובה: ב** — 256 כתובות, פחות כתובת הרשת וכתובת ה-254 Broadcast.

3.9 פרוטוקולי תשתית

DNS — Domain Name System

מתרגם שם דומיין (למשל example.com) לכתובת IP.

- עובד בעיקר מעל UDP, פורט 53.
- יעד מרכזי ל-OSINT ולתקיפות כמו DNS Spoofing ו-Zone Transfer.

DHCP — Dynamic Host Configuration Protocol

מחלק כתובות IP אוטומטית למכשירים חדשים ברשת — יחד עם כתובת ה-Gateway ושרת ה-DNS.

NAT — Network Address Translation

הראוטר "מתרגם" כתובות פרטיות רבות לכתובת ציבורית אחת. זו הסיבה שלכל המכשירים בבית יש כתובת 192.168.x.x, אך כלפי האינטרנט מופיעה כתובת ציבורית אחת בלבד.

ICMP — Internet Control Message Protocol

פרוטוקול בקרה. הפקודה `ping` משתמשת בו כדי לבדוק אם מכונה "חיה".

Zone Transfer (הפקודה `AXFR`): אם שרת ה-DNS מוגדר שגוי, אפשר "לשאוב" ממנו את כל רשומות הדומיין — פרס ב-Recon:

```
dig axfr @nsserver target.com
```

הדגמה — נסה בעצמך: ארבעת פרוטוקולי התשתית

DNS — תרגום שם ל-IP (דורש NAT):

```
dig google.com +short      # של הדומיין IP-ה
dig google.com MX +short   # שרתי הדואר שלו
```

שם → כתובת IP.

DHCP — מאיפה קיבלת כתובת:

```
ip a
```

ה-IP שלך לא הוגדר ידנית — **DHCP** חילק אותו אוטומטית.

NAT — פרטי מול ציבורי (דורש NAT):

```
echo "פרטי:"; ip -4 a | grep inet | grep -v 127
echo "ציבורי:"; curl -s ifconfig.me; echo
```

שתי כתובות שונות — התרגום בפעולה.

ICMP — הפרוטוקול של ping:

```
sudo tcpdump -i any -n icmp # 1 טרמינל
ping -c3 8.8.8.8             # 2 (NAT דורש) טרמינל
```

ICMP echo request → ICMP echo reply — זה בדיוק מה ש-`ping` עושה.

מה קרה כאן: ראית את ארבעת פרוטוקולי התשתית שעובדים "מאחורי הקלעים" בכל גלישה. 🎯

🧠 **בדוק את עצמך:** הפקודה `ping` משתמשת באיזה פרוטוקול? - (א - TCP - ב - UDP - ג - ICMP)

✅ **תשובה: ג** — `ping` שולח ICMP echo request ומקבל `echo reply`.

3.10 כלים מעשיים

<code>ip a</code>	# IP ו-MAC כתובות
<code>ip route</code>	# Gateway ה
<code>ping -c 4 8.8.8.8</code>	# קישוריות (ICMP)
<code>traceroute google.com</code>	# מסלול החבילה (צמתים)
<code>ss -tuln</code>	# פורטים מאזינים מקומית
<code>nslookup google.com</code>	# DNS תרגום
<code>dig google.com</code>	# מפורטת DNS שאילתת
<code>ipcalc 192.168.1.0/24</code>	# חישוב תת-רשת

Wireshark ו-tcpdump — לראות את החבילות

- **Wireshark** — כלי גרפי ללכידת וניתוח תעבורה. מסננים שימושיים: `tcp.flags.syn==1`, `http`, `dns`.
- **tcpdump** — גרסת שורת הפקודה: `sudo tcpdump -i eth0 port 80`.

💡 **תרגיל מנטלי:** פתח Wireshark, גלוש לאתר, וזהה את רצף ה-ACK → SYN-ACK → SYN. לראות את הלחיצה "בחיים" מקבע את ההבנה.

🎬 הדגמה — נסה בעצמך: Wireshark — לראות הכול

הפיינל: כלי גרפי שמראה כל חבילה. ודא ששרת ה-Flask רץ.

צעד 1 — פתח Wireshark:

```
sudo wireshark &
```

צעד 2 — התחל לכידה: לחיצה כפולה על הממשק `lo` (loopback).

צעד 3 — סנן רק את התעבורה שלנו: בשורת הסינון למעלה הקלד:

```
tcp.port == 5000
```

```
curl http://127.0.0.1:5000
```

👁️ מה תראה: את לחיצת היד (SYN, SYN-ACK, ACK) ואז את בקשת/תשובת ה-HTTP.

צעד 5 — ראה את כל השיחה: לחיצה ימנית על חבילה → **TCP Stream** → **Follow**. 👁️ כל השיחה בין הדפדפן לשרת, בעין אחת.

🎯 מה קרה כאן: Wireshark הוא "צילום רנטגן" לרשת — כל מה שלמדנו בפרק (שכבות, לחיצת יד, פורטים, HTTP) נראה כאן יחד.

🏆 המשך לאתגר הגדול ב- [missions.md](#): "ציד החבילות" — לכווד סיסמה שרצה בטקסט גלוי.

🧠 בדוק את עצמך: מה עושה **Follow** → **TCP Stream** ב-Wireshark? (א) סוגר את החיבור - (ב) מציג את כל השיחה בין שני הצדדים בתצוגה אחת - (ג) סורק פורטים

✅ תשובה: ב — הוא מרכז את כל החבילות של אותו חיבור לתצוגה קריאה אחת.

סיכום המודול

- מסע חבילה: HTTP → TCP Handshake → ARP → DNS; כל שלב הוא שכבה ונקודת תקיפה.
- OSI (7) ו-TCP/IP (4) מתארים את התקשורת; שכבה 4 = פורטים, שכבה 3 = IP, שכבה 2 = MAC.
- TCP אמין (Handshake) · UDP מהיר; תשובת SYN קובעת אם פורט פתוח/סגור/מסונן.
- IP (פרטי/ציבורי), MAC/ARP, פורטים ו-Subnetting/CIDR מגדירים את משטח התקיפה.
- DNS/DHCP/NAT/ICMP הם התשתית — וגם יעדים (Spoofing, Zone Transfer).

מה הלאה?

→ תרגילים — [missions.md](#) · פתרונות — [solutions.md](#) · תרגול ותרחישים — [practice.md](#) →

מודול 4 — סקריפטינג: Bash ו-Python

← חזרה למפת הקורס